```
  def getBetByMarket(
      in: MarketIdsBet): scala.concurrent.Future[SumStakes] = {     ❶
    Future {
      ScalikeJdbcSession.withSession { session =>
        val sumStakes = betRepository
          .getBetPerMarketTotalStake(in.marketId, session)          ❷
          .map { each =>
            SumStake(each.sum, each.result)
          }
        SumStakes(sumStakes)
      }
   }}
  ...
}
```

❶ Input from the RPC call

❷ Calls to the repository to retrieve the data

NOTE The future in the output indicates that the result will come back at some point, and when that happens, a thread will be used. You could use the same thread pool from the `ActorSystem`, but since the connection to the database is a blocking call, you should use a different thread pool. Otherwise, you can block the `ActorSystem` so that it can no longer run actors, send messages, and function as expected.

The following snippet shows how to use `akka.projection.jdbc.blocking-jdbc-dispatcher` from the `akka-projection-jdbc` module along with the rest of the service implementation:

```
class BetProjectionServiceImpl(
    system: ActorSystem[_],
    betRepository: BetRepository)
    extends BetProjectionService {

  implicit private val jdbcExecutor: ExecutionContext =
    system.dispatchers.lookup(
      DispatcherSelector.fromConfig(
```